Answer Key

Q1 — **B.** `logic` is a 4-state type (can hold 0, 1, X, Z); `bit` is a 2-state type (only 0 or 1). Choice A reverses them.

Q2 — **B.** When `-5` is assigned to an unsigned `int`, it wraps to `4294967291` (two's complement interpretation). Signed `s` stays `-5`.

Q3 — **B.** `struct` allocates memory for all members independently; `union` shares memory among members (only one can be used at a time).

Q4 — **A.** `byte` is signed; when assigned `-1`, it remains `-1`. `int` is also signed, so `-1` is `-1`. Both print `-1`.

Q5 — **B.** The `exists()` method checks if a key exists in an associative array. Since `"key2"` was never assigned, it doesn't exist.

Q6 — **A.** The `find()` method with a `with` clause filters elements matching the condition `x > 3`, which are `4` and `5`.

Q7 — **B.** `obj` is a null handle (not initialized with `new()`). Accessing `obj.value` causes a null pointer dereference error at runtime.

Q8 — **B.** Virtual methods enable polymorphism. Even though `b` is declared as `Base`, it holds a `Derived` object, so `Derived::show()` is called.

Q9 — **C.** Static members are shared across all instances. Each `new()` increments `count`, so after two instantiations, `count = 2`.

Q10 — **A.** The `$cast()` succeeds because `b` actually holds a `Derived` object. `d` is now a valid reference, and `d.x = 10` is printed.

Q11 — **A.** `solve...before` establishes a randomization order, allowing later constraints to reference the value of earlier-randomized variables.

Q12 — **A, C** In `dist`, the weights are relative probabilities. The total weight is 100 (70 + 20 + 10), so `level == 1` has probability 70/100 = 70%.

Q13 — **C.**

Q14 — **B.** The loop prints `0` and `1`. At `i == 2`, `continue` skips to the next iteration. At `i == 4`, `break` exits the loop.

Q15 — **A.** Pass-by-value means changes to `val` inside `modify_val()` don't affect the original `x`. `x` remains `5`.